

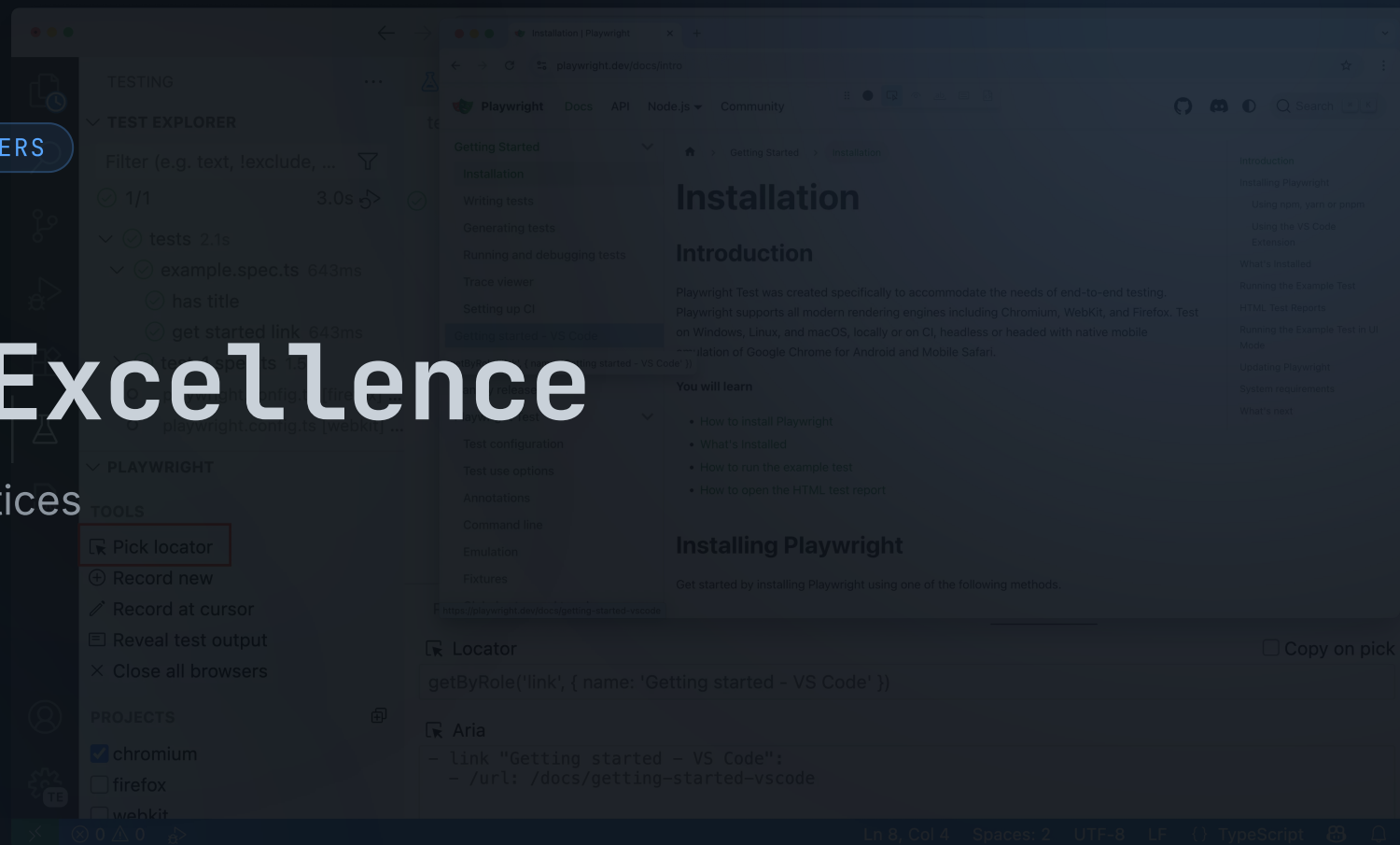
4-SESSION TRAINING PROGRAM FOR QA ENGINEERS

Playwright Automation Excellence

Session 1: Foundations & Best Practices



By Utain W.
Tech Lead



Agenda

01

Foundations

TypeScript, npm & Module System

02

Strategy

When & What to Automate, Prioritization

03

Vocabulary

Playwright & QA Domain Glossary

04

Test Techniques

Functional vs Non-Functional, Where
Playwright Fits

05

UI Testing

Best Practices, POM, Flaky Prevention

06

API & Architecture

API Testing, Refactoring & Code Structure

01 / 06

Foundations

TypeScript, npm & Module System — The building blocks of solid
test automation

TypeScript Essentials for QA

Why TypeScript?

Type safety catches bugs at **compile time**, not runtime. IDE autocompletion speeds up development.

Key Concepts to Know

interfaces — define shape of test data

types — string, number, boolean, enum

async/await — every Playwright action is async

generics — reusable utility functions

Rule of Thumb

If your test file has any **any** type, you're doing it wrong.

```
login.utils.ts

interface LoginData {
  username: string;
  password: string;
}

async function login(data: LoginData) {
  await page.fill('#user', data.username);
  await page.fill('#pass', data.password);
  await page.click('#submit');
}
```

npm & ES Module System

npm (Node Package Manager)

Installs Playwright, manages dependencies, runs scripts via package.json

Key Commands

```
npm init -y
```

 scaffold project

```
npm install -D @playwright/test
```

 install Playwright

```
npx playwright install
```

 install browsers

ES Modules – import/export

Organize code into reusable modules. Each file is a module. Use named exports for page objects, utilities, and test data.

Why This Matters

Modular code = maintainable code. No 1000-line test files.

```
utils/helpers.ts

// utils/helpers.ts
export function generateEmail(): string {
  return `test_${Date.now()}@qa.dev`;
}
```

```
tests/login.spec.ts

// tests/login.spec.ts
import { test, expect } from '@playwright/test';
import { generateEmail } from '../utils/helpers';
```

02 / 06

Strategy

When & What to Automate — Not everything deserves a test script

When to Automate **vs.** When NOT To

✅ **AUTOMATE These**

- Repetitive regression scenarios run every sprint
- Critical user journeys (login, checkout, core flows)
- Data-driven tests with many input combinations
- API contract validation between services
- Cross-browser compatibility checks

❌ **Do NOT Automate These**

- One-time exploratory testing
- UX subjective evaluations (look & feel)
- Features under active redesign (waste of effort)
- Tests with unstable test data dependencies
- Scenarios requiring human judgment (accessibility nuance)

💡 **Theory:** Every automated test has a maintenance cost. Automate when the value of repeated execution exceeds the cost of writing and maintaining the script.

Test Automation Priority Matrix

IMPACT →

! DO FIRST

High Impact + Stable Feature

- Core user journeys (login, checkout, payment)
- API smoke tests
- Critical path E2E

🕒 SCHEDULE

High Impact + Unstable Feature

- Write test plan only
- Mark for next sprint automation
- Use API tests instead of UI for unstable areas

👤 DELEGATE

Low Impact + Stable Feature

- Use existing manual test cases
- Add to regression checklist
- Automate only if time permits

⊘ SKIP

Low Impact + Unstable Feature

- Do not invest automation effort here
- Manual ad-hoc testing only

📌 When timeline is tight: Always protect the DO FIRST quadrant. Automation is an investment — invest where ROI is highest.

03 / 06

Vocabulary

Playwright & QA Domain Glossary — Speak the same
language

Playwright & QA Glossary



Locator

Playwright's auto-waiting element reference. Replaces CSS selectors.

```
page.getByRole('button', { name: 'Submit' })
```



Fixture

Reusable test setup/teardown context in Playwright. Runs before/after tests.

Example: authenticated page, test database.



Assertion

Verification that actual result matches expected. Playwright uses expect() with auto-retry.

```
expect(button).toBeVisible()
```



POM

Page Object Model — Design pattern: one class per page, encapsulating selectors and actions.

Separates test logic from page structure.



Flaky Test

A test that passes and fails without code changes. The **#1 enemy** of test reliability.

Destroys trust in your test suite.



Test Isolation

Each test runs in a clean state. No shared state between tests. Default in Playwright.

Never disable this default behavior.

04 / 06

Test Techniques

Functional vs Non-Functional — Know the landscape
before you automate

04

Testing Techniques – The Full Landscape

🔗 Non-Functional Testing

- **Load Testing**
How the system handles expected volume
- **Performance Testing**
Response time under normal load
- **Stress Testing**
Behavior beyond normal capacity
- **Security Testing**
Vulnerabilities & penetration checks
- **Accessibility Testing**
WCAG compliance & usability for all

✅ Functional Testing

- **Unit Testing**
Individual functions in isolation
- **Integration Testing**
Interactions between modules/services
- **Mocking**
Simulating dependencies for isolated testing
- **Smoke Testing**
Quick check: does the build work at all?
- **Sanity Testing**
Quick check: does a specific area still work?
- **Regression Testing**
Verify existing features aren't broken
- **Exploratory Testing**
Unscripted discovery of unknown issues
- **UAT (User Acceptance Testing)**
Business validation by stakeholders



Theory: Each technique serves a different purpose. A mature QA strategy uses a combination — not just one type.

Where Playwright Fits – Automation Coverage Map

✓ EXCELLENT Fit	⚡ PARTIAL Fit	⊘ NOT for Playwright
Automate with Playwright > Integration Testing API + UI flow > Smoke Testing Quick build validation > Sanity Testing Targeted area checks > Regression Testing Repeat every sprint > Mocking Intercept & modify network > Accessibility Testing Automated WCAG checks via axe-core	Playwright + Other Tools > Unit Testing Use Jest/Vitest instead > Load Testing Use k6/Artillery + Playwright for realism > Performance Testing Use Lighthouse + Playwright traces	Use dedicated tools > Exploratory Testing Requires human curiosity > UAT Requires business stakeholder judgment > Security Testing Use OWASP ZAP/Burp Suite > Stress Testing Needs dedicated infrastructure tools



Key Insight: Playwright is your automation workhorse for Functional + Accessibility testing. Know its strengths and limitations to set realistic expectations with your team.

05 / 06

Playwright for UI Testing

Best Practices — Write tests that are reliable, readable, and maintainable

UI Testing Best Practices



Use Locators, Not Selectors

Prefer `getByRole`, `getByText`, `getByLabel` over CSS/XPath.

Locators are resilient to DOM changes and auto-wait. CSS selectors break when styling changes.



Assert Visibly, Not Internally

Use `toBeVisible()`, `toHaveText()` — assertions users can verify.

Avoid `toHaveAttribute('class', 'active')` — implementation detail.



Never Use Hard Waits

No `page.waitForTimeout(5000)`. Ever.

Use `page.waitForSelector()` or let Playwright auto-wait via locators. Hard waits = flaky tests.



One Concern Per Test

Each test validates **one behavior**. Don't test login + profile update + logout in one test.

Failure becomes unclear. Maintenance becomes nightmare.

Page Object Model (POM)



What

One class per page/component. Encapsulates selectors and actions.



Why

When UI changes, update **ONE** place, not 50 test files.



When

Always. Even for simple pages. Habit beats cleverness.



Rule

Test files should read like a **user story**, not a DOM map.

```
// pages/login.page.ts

export class LoginPage {
  constructor(private page: Page) {}
  private get usernameInput() {
    return this.page.getByLabel('Username');
  }
  private get submitBtn() {
    return this.page.getByRole('button', { name: 'Login' });
  }
  async login(user: string, pass: string) {
    await this.usernameInput.fill(user);
    await this.page.getByLabel('Password').fill(pass);
    await this.submitBtn.click();
  }
}
```

💡 Test file reads: `await loginPage.login(user, pass)` — clean and readable

Flaky Test Prevention

Kill the #1 Enemy of Test Reliability



DO

Proper Waiting

- ▶ Let Playwright auto-wait via locators
- ▶ Use `waitForURL()` for navigation
- ▶ Use `waitForResponse()` for API calls
- ▶ Use `toBeVisible()` assertions that auto-retry



AVOID

Common Anti-Patterns

- ✗ `page.waitForTimeout()` — hard waits hide timing bugs
- ✗ Relying on `sleep/delay` for async operations
- ✗ Assuming element is ready without assertion
- ✗ Sharing state between tests



CONFIGURE

Retries Wisely

- ⚙ Set retries: 2 in `playwright.config.ts` for CI
- ⚙ Set retries: 0 locally (catch real issues)
- ⚙ Use trace: 'on-first-retry' to debug failures
- ⚙ Never use retries to hide real flakiness

06 / 06

API Testing & Architecture

API tests for stability, architecture for maintainability

Playwright API Testing Best Practices



Why API Tests First

10x faster than UI, no browser needed, catches backend bugs early



Use Request Context

Playwright's APIRequestContext sends HTTP requests without browser overhead



Test the Contract

Validate status codes, response schema, and error messages



Chain APIs

Use response data from one call as input to the next — real user flow simulation



Isolate Test Data

Create fresh data via API before each test, clean up after

```
import { test, expect } from '@playwright/test';
test('create and verify user', async ({ request }) => {
  const response = await request.post('/api/users', {
    data: { name: 'QA Test', email: 'qa@test.dev' }
  });
  expect(response.ok()).toBeTruthy();
  expect(response.status()).toBe(201);
  const user = await response.json();
  expect(user.name).toBe('QA Test');
});
```

Code Architecture & Refactoring

📁 Folder Structure

```
playwright-project/  
  pages/           ← Page Object classes  
  tests/  
    ui/           ← UI test specs  
    api/          ← API test specs  
  fixtures/       ← Custom test fixtures  
  utils/          ← Helper functions  
  data/           ← Test data (JSON)  
  playwright.config.ts
```

✂️ Refactoring Rules



Extract, Don't Duplicate

Same logic in 2+ tests? Extract to a utility or fixture. DRY principle applies to tests too.



Compose, Don't Inherit

Use Playwright fixtures to compose test setup. Avoid deep inheritance chains — they create fragile coupling.



Delete Dead Code Ruthlessly

Unused page objects, skipped tests, commented-out assertions — delete them. They mislead and confuse.

Homework & Next Session Preview



Homework

1. Set up a Playwright project from scratch using TypeScript and the folder structure from today
2. Write 3 UI tests for any public website using Page Object Model
3. Write 2 API tests using Playwright request context
4. **Bonus:** Identify 2 flaky tests in your current suite and apply today's prevention strategies



Next Session Preview

- > Parallel Execution & Test Sharding
- > Visual Regression Testing
- > CI/CD Pipeline Integration
- > Test Reporting & Metrics